

Queue (Hàng đợi): Giải thích, sơ đồ ASCII và mã Python minh họa

Sơ đồ minh họa (ASCII):

Trạng thái ban đầu:

```
[ ] [ ] [ ] [ ] [ ] (Queue trống)
```

Sau khi Enqueue 10, 20, 30:

```
Front -> [10] [20] [30] <- Rear
```

Dequeue (lấy phần tử đầu):

```
Front -> [20] [30] <- Rear
```

Nguyên tắc: Thêm vào Rear (cuối), Lấy ra từ Front (đầu).

Phần 1: Queue với collections.deque (khuyến nghị)

Mã nguồn:

```
from collections import deque

# Khởi tạo queue rỗng
q = deque()

# Enqueue: thêm vào cuối (rear)
q.append(10)
q.append(20)
q.append(30)
print('Sau khi enqueue 10, 20, 30:', list(q)) # [10, 20, 30]

# Dequeue: lấy từ đầu (front)
x = q.popleft()
print('Dequeue:', x) # 10
print('Trạng thái queue:', list(q)) # [20, 30]

# Peek front
front = q[0] if q else None
print('Front (peek):', front) # 20

# Kiểm tra rỗng
print('Queue rỗng?', len(q) == 0)
```

```
# Duyệt queue theo FIFO
for item in q:
    print('Duyệt:', item)
```

Giải thích nhanh:

- `append(x)`: Thêm vào rear với chi phí $O(1)$.
- `popleft()`: Lấy từ front với chi phí $O(1)$.
- deque tối ưu cho thao tác hai đầu, phù hợp cài đặt Queue.

Phần 2: Queue với list (đơn giản nhưng kém hiệu quả cho dequeue)

Mã nguồn:

```
# KHÔNG khuyến nghị cho dữ liệu lớn vì  $\text{pop}(0) = O(n)$ 
q = []
```

```
# Enqueue
q.append(10)
q.append(20)
q.append(30)
print('Sau khi enqueue:', q) # [10, 20, 30]
```

```
# Dequeue (chậm với list lớn vì phải dịch phần tử)
x = q.pop(0)
print('Dequeue:', x) # 10
print('Trạng thái queue:', q) # [20, 30]
```

```
# Peek front
front = q[0] if q else None
print('Front (peek):', front) # 20
```

Ghi chú:

- `pop(0)` phải dịch toàn bộ phần tử sang trái $\rightarrow O(n)$.

Phần 3: Tự cài đặt Circular Queue (mảng vòng)

Mã nguồn:

```
class CircularQueue:
    def __init__(self, capacity):
        self.capacity = capacity
        self.data = [None] * capacity
        self.size = 0
        self.front = 0
```

```

        self.rear = -1

    def is_empty(self):
        return self.size == 0

    def is_full(self):
        return self.size == self.capacity

    def enqueue(self, x):
        if self.is_full():
            raise OverflowError('Queue đầy')
        self.rear = (self.rear + 1) % self.capacity
        self.data[self.rear] = x
        self.size += 1

    def dequeue(self):
        if self.is_empty():
            raise IndexError('Queue rỗng')
        x = self.data[self.front]
        self.data[self.front] = None
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return x

    def peek_front(self):
        return None if self.is_empty() else self.data[self.front]

    def __repr__(self):
        out = []
        idx = self.front
        for _ in range(self.size):
            out.append(self.data[idx])
            idx = (idx + 1) % self.capacity
        return f'CircularQueue({out})'

# Minh họa:
cq = CircularQueue(capacity=5)
cq.enqueue(10); cq.enqueue(20); cq.enqueue(30)
print('Sau enqueue 10,20,30:', cq)
print('Dequeue:', cq.dequeue())
print('Sau dequeue:', cq)
cq.enqueue(40); cq.enqueue(50)
print('Sau enqueue 40,50:', cq)
print('Peek front:', cq.peek_front())

```

Điểm chính:

- Circular Queue dùng mảng cố định và quay vòng chỉ số front/rear bằng phép % capacity.
- enqueue/dequeue đều $O(1)$, phù hợp cho buffer dung lượng cố định.